PLS bei nicht erreichbarem Server

Ist-Analyse
Paket C · Stufe 1 · Ebene 1: Gerät

Marlene Sakura Heinrich
FH-Projekt Johanniter / PLS–TRIAS Integration

Zusammenfassung

Die Untersuchung bewertet das bestehende Verhalten von PLS bei fehlender Verbindung zum PLS-Server. Grundlage sind praktische Ausfalltests, die Analyse relevanter Frontend-Dateien sowie vorhandene automatisierte Tests. Es wurden keine neuen Offline-Funktionen implementiert.

Bereits geladene Ansichten bleiben teilweise nutzbar. Triageentscheidungen und Körpermarkierungen können dabei sichtbar bestätigt erscheinen, obwohl der Serveraufruf fehlschlägt. Die geprüften Änderungen wurden nicht dauerhaft gespeichert und nach Wiederverbindung nicht automatisch nachgesendet.

Für Seite 1 des Ambulanzprotokolls besteht bereits eine lokale Speicherung in localStorage. Ein Entwurf blieb über einen vollständigen Browser-Neustart erhalten und wurde nach Wiederherstellung des Servers erneut geladen und übertragen. Die Anwendung selbst konnte ohne Server nach Reload oder Neustart jedoch nicht geöffnet werden.

Damit besteht eine teilweise Offline-Funktionalität für Formularentwürfe, jedoch keine durchgängig nachgewiesene Offline-Triage mit dauerhafter Speicherung und zuverlässiger späterer Übertragung.

1. Ziel und Untersuchungsumfang

Ziel von Paket C, Stufe 1, ist eine Bestandsaufnahme: Welche Funktionen von PLS funktionieren auf dem Gerät ohne erreichbaren PLS-Server? Das Ergebnis wird in einer Offline-Funktionstabelle mit Begründungen und nachvollziehbaren Testabläufen dokumentiert.

Untersucht wurden Leitstellenübersicht, Triageentscheidungen, Körpermarkierungen, Ambulanzprotokoll Seite 1, Browser-Speicher sowie das Verhalten bei Serverausfall, Reload, Browser-Neustart und Wiederverbindung. Das Ambulanzprotokoll gehört hier zur untersuchten PLS-Anwendung; die separate TRIAS-Anwendung wurde nicht auf Offline-Funktionalität getestet.

Die Verbindung zwischen PLS-Server und Ambulanzsystem gehört zu Paket D und liegt außerhalb dieser Untersuchung. Die Implementierung einer IndexedDB-Warteschlange, eines Service Workers oder einer Offline-Anmeldung ist ebenfalls nicht Bestandteil der Ist-Analyse.

2. Testumgebung und Vorgehen

Merkmal	Testumgebung
Anwendung	Lokales PLS; .NET-8-Backend, React-Frontend, MySQL und Mosquitto
Browser / Werkzeuge	Safari auf macOS; IntelliJ und Terminal
Aufruf	http://localhost:8080
Betriebsart	Docker Compose: docker-compose.yml + docker-compose.dev.yml
Docker-Versionen	Engine 29.8.1; Compose v5.5.1

Merkmal	Testumgebung
Git-Branch	C-s1-offline-ist-analyse
Lokaler Quellcode-Stand	2960f2a20efda1b3351b7dac34d64f412e058fdf
Testkontext	Lokaler Einsatz IKS; Rollen Leitstelle und Triage; Testpatienten 5, 6 und 7

Der dokumentierte Commit bezeichnet den lokalen Quellcode-Stand. Eine eindeutige Zuordnung des laufenden Docker-Images zu diesem Commit wurde nicht erhoben. Die praktischen Ergebnisse beziehen sich auf die tatsächlich ausgeführte lokale Anwendung.

Simulation des Serverausfalls

„Offline“ bezeichnet in dieser Untersuchung die fehlende Erreichbarkeit des PLS-Anwendungsservers. Dazu wurde ausschließlich der Container pls-app gestoppt. Da dieser sowohl die API als auch die Frontend-Dateien bereitstellt, fehlen während des Ausfalls auch neu abzurufende Anwendungsdateien. Die Datenbank und der MQTT-Dienst werden dadurch nicht gelöscht oder gezielt gestoppt.

Der Browser-Netzwerkstatus wurde getrennt von der Server-Erreichbarkeit betrachtet. navigator.onLine blieb bei bestätigtem Serverausfall true. Eine Internet- oder WLAN-Unterbrechung allein würde die Erreichbarkeit von localhost nicht zuverlässig unterbinden.

Schritt	Befehl / Kontrolle
Anwendung starten	docker compose -f docker-compose.yml -f docker-compose.dev.yml start pls-app
Erreichbarkeit prüfen	curl --max-time 5 -i http://localhost:8080/health Erwartung: HTTP 200 und Healthy
Testzustand vorbereiten	Betroffene Seite und Testpatient online öffnen; Ausgangszustand festhalten.
Server stoppen	docker compose -f docker-compose.yml -f docker-compose.dev.yml stop pls-app
Ausfall bestätigen	Health-Aufruf wiederholen; erwarteter Verbindungsfehler: curl (7).
Aktion prüfen	Eine Eingabe ausführen; Oberfläche, Konsole und gegebenenfalls Browser-Speicher kontrollieren.
Wiederverbindung prüfen	pls-app starten und HTTP 200 abwarten; Serverdatensatz erneut lesen. Bei Tests auf automatische Übertragung zunächst nicht bearbeiten oder neu laden.

Die Bewertung unterscheidet zwischen sichtbarer Änderung, dauerhafter lokaler Speicherung und bestätigter Speicherung am Server. Codebefunde und praktisch beobachtete Ergebnisse werden getrennt ausgewiesen.

3. Offline-Funktionstabelle

Die Angaben gelten für den beschriebenen Testumfang. „Ja, vorübergehend“ bezeichnet ausschließlich die Anzeige im geöffneten Browser und keine dauerhafte Speicherung. C01–C12 verweisen auf die Testprotokolle in Abschnitt 4.

Funktion	Funktioniert offline ja/nein	Grund
Bereits geladene Leitstellenübersicht weiter anzeigen	Ja, eingeschränkt	Die vorhandenen Zeilen blieben sichtbar. Damit sind keine aktuellen Daten oder erneuten Abrufe nachgewiesen. Praktisch: C01.
Anwendung nach Reload oder Browser-Neustart ohne Server öffnen	Nein	Safari konnte die Seite nicht laden. Anwendungsdateien waren nicht erreichbar. Praktisch: C01, C03, C08.
Triageauswahl auf bereits geöffneter Seite anzeigen	Ja, nur vorübergehend	GRÜN/GELB wurden angezeigt, bevor der Server die Änderung bestätigte. Kein Speichernachweis. Praktisch: C08, C10.
Triagefarbe dauerhaft speichern und später übertragen	Nein, in den geprüften Fällen	API-Aufruf scheiterte; Serverwerte blieben null. Keine Nachsendung beobachtet. Praktisch: C08, C10; Code: usePatientTriage.
Atmung dauerhaft speichern	Nein, im geprüften Fall	Die Anwendung ging weiter zur nächsten Seite; die API schlug fehl und atmung blieb am Server null. Praktisch: C09.
Blutungsentscheidung dauerhaft speichern	Nein, im geprüften Fall	GELB erschien lokal; blutung und triagefarbe blieben am Server null. Praktisch: C10.
Körpermarkierung auf bereits geladener Grafik anzeigen	Ja, nur vorübergehend	Kopfmarkierung wurde sichtbar; Speichern schlug fehl. Nach erneutem Öffnen war sie verschwunden. Praktisch: C11.
Körpermarkierung dauerhaft speichern / später senden	Nein, im geprüften Fall	kopf_vorne blieb am Server 0. Keine Nachsendung bis zur Kontrolle. Praktisch: C11.
Körpergrafik neu öffnen und Körperteile laden	Nein, im geprüften Fall	Der GET-Aufruf scheiterte; die Seite zeigte keine Körpergrafik. Praktisch: C12.
Ambulanzprotokoll Seite 1 auf geöffneter Seite bearbeiten und lokal sichern	Ja	Testtext wurde in localStorage abgelegt; Anzeige saved-local. Praktisch: C02; Code: page1Storage.
Lokalen Formularentwurf über Browser-Neustart behalten	Ja, hinsichtlich Speicherung	Der Text war nach Rückkehr des Servers wieder vorhanden. Offline konnte die Anwendung nach Neustart nicht geöffnet werden. Praktisch: C03.
Lokalen Formularentwurf nach Rückkehr des Servers mit Reload übertragen	Ja, im geprüften Ablauf	Danach saved-server; Serverabfrage bestätigte den Testtext. Praktisch: C04, C05.
Formular bei bloßer Rückkehr des PLS-Servers automatisch übertragen	Nein, innerhalb der beobachteten 30 Sekunden	Ohne Reload, weitere Eingabe oder online-Ereignis blieb saved-local; Server enthielt den alten Text. Praktisch: C05.
Formular nach ausgelöstem online-Ereignis übertragen	Ja, bedingt	Ein manuell ausgelöstes Browserereignis führte zur bestätigten Speicherung. Eine echte Netzunterbrechung wurde damit nicht getestet. Praktisch: C06.
navigator.onLine als Nachweis der PLS-Erreichbarkeit verwenden	Nein	Der Wert blieb true, obwohl curl den Server nicht erreichen konnte. Praktisch: C07.

Ergänzende Befunde aus der Codeanalyse

Die folgenden Abläufe wurden anhand des Codes bewertet, jedoch nicht vollständig praktisch unter Serverausfall geprüft.

Funktion	Bewertung aus dem Code	Grund / Grenze
Neue Anmeldung ohne erreichbaren Server	Nein für die gezeigten Login-Aufrufe	Sie verwenden Server-POSTs. Bestehende Tokens liegen lokal, beweisen aber keine Offline-Anmeldung. Kein separater Offline-Login-Test.
Neuen Patienten über QR-Code bestätigen / anlegen	Nein für den gezeigten Ablauf	ScanPatient erwartet verifyPatientQrCode vom Server vor der Navigation. Online-Testhilfe genutzt; kein vollständiger Offline-Scan mit Kamera durchgeführt.
Auth-Token lokal behalten	Ja, Speicher vorhanden	auth-storage persistiert Token und Admin-Token. Tokenwerte werden hier nicht wiedergegeben. Validierung benötigt weiterhin die API.
GPS-Auftragszustand lokal speichern	Ja, Speicher vorhanden	patient-location-storage persistiert jobs. Tatsächliche Koordinatenaufnahme und Offline-Übertragung wurden wegen verweigertem GPS-Zugriff nicht nachgewiesen.
GPS-Fehler automatisch erneut senden	Nein im gezeigten Store-Ablauf	Ein vorhandener error-Job wird von ensurePatientLocation nicht erneut gestartet. Keine automatische Übertragung bei online erkennbar; kein vollständiger praktischer GPS-Test.

4. Praktische Testprotokolle

C01 – Geladene Übersicht und Reload

Durchführung: Leitstellenübersicht mit fünf Patienten online laden, pls-app stoppen und Verbindung prüfen. Geladene Ansicht beobachten; anschließend Seite neu laden.

Ergebnis: Bereits sichtbare Daten blieben zunächst erhalten. Beim Reload ohne Server konnte Safari die Seite nicht öffnen. Nach Wiederherstellung erschien zeitweise „Keine Personen gefunden“; nach erneuter Auswahl von IKS / Leitstelle waren die fünf Patienten wieder sichtbar.

Bedeutung: Geladene Bildschirmdaten sind weiter nutzbar, aber keine belastbare Offline-Datenquelle. Die leere Ansicht belegt keinen Verlust der Datenbank; der Auswahlkontext ist gesondert zu prüfen.

C02 – Formular offline lokal speichern

Durchführung: Ambulanzprotokoll Seite 1 für Patient 5 online öffnen. Server stoppen. Ambulanzort auf einen eindeutigen Testtext ändern; Speicherstatus und localStorage kontrollieren.

Ergebnis: saved-local; der Entwurf war lokal vorhanden.

Bedeutung: Diese Formularfunktion hat bereits eine lokale Speicherung. Eine Serverbestätigung liegt zu diesem Zeitpunkt nicht vor.

C03 – Formularentwurf und Browser-Neustart

Durchführung: Nach C02 Safari mit ⌘ Q vollständig beenden und bei weiterhin gestopptem Server wieder öffnen. Danach Server starten, HTTP 200 abwarten und Formular erneut laden.

Ergebnis: Ohne Server war die Formularseite nicht erreichbar. Nach Serverrückkehr war der Testtext vorhanden; anschließend saved-server. Die Serverabfrage bestätigte den Text.

Bedeutung: Der lokale Entwurf überstand den Neustart. Die Offline-Nutzbarkeit der gesamten Anwendung überstand ihn nicht.

C04 – Übertragung nach Wiederverbindung mit Reload

Durchführung: Server stoppen, Formulartext auf OFFLINE-TEST-C-S1-RECONNECT ändern und saved-local abwarten. Server starten; nach Healthy Seite neu laden und gespeicherten Datensatz lesen.

Ergebnis: Nach Reload saved-server. Server enthielt OFFLINE-TEST-C-S1-RECONNECT.

Bedeutung: Die Wiederaufnahme mit Reload funktioniert im getesteten Formularablauf. Daraus folgt noch keine automatische Nachsendung ohne Reload.

C05 – Rückkehr des Servers ohne weitere Aktion

Durchführung: Online OFFLINE-TEST-C-S1-AUTO-BASE speichern. Server stoppen, auf OFFLINE-TEST-C-S1-AUTO ändern. Server wieder starten; etwa 30 Sekunden nicht bearbeiten und nicht neu laden. Serverdatensatz kontrollieren. Danach neu laden und erneut kontrollieren.

Ergebnis: Zunächst blieb saved-local; Server enthielt weiter AUTO-BASE. Erst nach Reload wurde AUTO übertragen und mit saved-server bestätigt.

Bedeutung: Bloße Serverrückkehr löste im beobachteten Zeitraum keine automatische Übertragung aus. Eine Aussage über einen längeren Zeitraum ist daraus nicht ableitbar.

C06 – Manuell ausgelöstes online-Ereignis

Durchführung: Server stoppen; OFFLINE-TEST-C-S1-ONLINE-EVENT lokal speichern. Server starten und Healthy abwarten. Ohne Reload in der Browserkonsole window.dispatchEvent(new Event('online')) ausführen. Serverdatensatz lesen.

Ergebnis: saved-server; Serverabfrage bestätigte den Text.

Bedeutung: Der vorhandene Ereignis-Handler kann einen ausstehenden Formularentwurf senden. Das künstliche Ereignis ist kein Test einer echten Netzunterbrechung.

C07 – Browser-Speicher und Erreichbarkeitsanzeige

Durchführung: Für localhost:8080 IndexedDB-Datenbanken, Service-Worker-Registrierungen sowie Schlüssel in localStorage und sessionStorage prüfen. Bei gestopptem Server navigator.onLine vergleichen.

Ergebnis: Keine IndexedDB-Datenbank und keine Service-Worker-Registrierung; sessionStorage leer. localStorage enthielt auth-storage und Formularentwürfe für Patienten 1–5, später zusätzlich patient-location-storage. navigator.onLine blieb trotz bestätigtem Serverausfall true.

Bedeutung: localStorage ist vorhanden. Der Browser-Netzwerkstatus allein reicht nicht für eine echte Server-Erreichbarkeitsprüfung. Speicherbefunde gelten für diesen Ursprung und Prüfzeitpunkt.

C08 – Triage GRÜN ohne Server

Durchführung: Patient 6 online vorbereiten. Server stoppen und curl-Fehler bestätigen. „Patient gehfähig“ wählen, Konsole prüfen. Browser-Neustart-/Reload-Verhalten untersuchen. Nach Wiederverbindung Personen des Einsatzes 1 am Server lesen.

Ergebnis: Die Oberfläche zeigte GRÜN. Der POST für Patient 6 scheiterte mit NetworkError. Nach Rückkehr war triagefarbe am Server null. Ohne Server war die Seite nicht erreichbar; nach Reload war die Patient-ID in der Triageansicht leer.

Bedeutung: Sichtbare Triagebestätigung ohne dauerhafte Speicherung. Keine erhaltene bzw. nachgesendete GRÜN-Änderung nachgewiesen. Der Patientenkontext wurde beim geprüften Reload nicht wiederhergestellt.

C09 – Atmung ohne Server

Durchführung: Patient 7 online auf TriagePage2 vorbereiten. Server stoppen und curl-Fehler bestätigen. „Atmung suffizient“ wählen. Nach Serverrückkehr den Patientendatensatz lesen.

Ergebnis: Die Anwendung ging weiter zur dritten Triage-Seite. Der respiration-POST scheiterte mit NetworkError. Nach Wiederverbindung war atmung am Server null.

Bedeutung: Die Navigation beweist keine Speicherung. Die Fehlerbehandlung unterbricht den aufrufenden Ablauf nicht.

C10 – Blutungsentscheidung und GELB ohne Server

Durchführung: Patient 7 auf TriagePage3 vorbereiten. Server stoppen, „Blutung Stillbar“ wählen und Konsole prüfen. Server wieder starten; nach dem Beobachtungszeitraum ohne erneute Eingabe den Datensatz lesen.

Ergebnis: GELB erschien. Der update-triage-color-POST scheiterte mit NetworkError. Nach Wiederverbindung waren triagefarbe und blutung am Server null.

Bedeutung: Die offline angezeigte Entscheidung wurde nicht bestätigt gespeichert oder bis zur Kontrolle nachgesendet. Im PLS-Code steht blutung=true für „stillbar“; diese Bedeutung darf nicht mit „Blutung vorhanden“ gleichgesetzt werden.

C11 – Körpermarkierung ohne Server

Durchführung: Körpervorderseite von Patient 7 online laden. Server stoppen; Kopf markieren. Nach Wiederverbindung zunächst ohne erneute Markierung den Serverwert kopf_vorne lesen. Danach zur Triage zurück und Körperseite erneut öffnen.

Ergebnis: Der Kopf wurde lokal rot angezeigt; PUT /save-body-part scheiterte mit NetworkError. Nach Wiederverbindung war kopf_vorne=0. Beim erneuten Öffnen war der Kopf wieder unmarkiert.

Bedeutung: Die Änderung blieb auf den aktuellen Ansichtsstatus beschränkt. Dauerhafte lokale Speicherung und automatische Nachsendung sind für diesen Ablauf nicht vorhanden.

C12 – Körpergrafik bei Ausfall neu laden

Durchführung: Server stoppen und aus der Triage die Körpervorderseite erneut öffnen. Anzeige und Konsole prüfen.

Ergebnis: „Front“ und Schaltflächen erschienen, aber keine Körpergrafik. GET /get-body-parts?idpatient=7 scheiterte mit NetworkError.

Bedeutung: Die Grafik wird aus den geladenen Körperteil-Einträgen aufgebaut. Bei fehlgeschlagenem Abruf bleiben diese leer.

Für einzelne Ausfallzyklen liegt kein gesonderter Health-Nachweis vor. Dort belegen die konkreten API-Verbindungsfehler den fehlgeschlagenen Zugriff. Der manuell ausgelöste online-Event testet den Ereignis-Handler, nicht eine tatsächliche Unterbrechung und Wiederherstellung des Netzwerks.

5. Technische Ursachen und Speichermechanismen

Die Codepfade beziehen sich auf PLS_Trias_Integration. Sie erklären die beobachteten Ergebnisse und grenzen vorhandene Speicherfunktionen von einer allgemeinen Offline-Warteschlange ab.

Codepfad	Relevanter Befund
client-app/src/api/client.ts	Axios nutzt /api. Tokens werden an Requests angehängt. HTTP 401 wird in der Konsole gemeldet; Fehler werden weitergereicht. In dieser Datei keine Offline-Warteschlange, kein eigener Retry und keine echte Erreichbarkeitsprüfung.
client-app/src/api/endpoints.ts	Login, Patientenabruf, QR-Verifikation, Triage, Atmung, Position und Körperteile verwenden direkte HTTP-Aufrufe. Die Aufrufe selbst sichern keine Offline-Änderungen.
client-app/src/hooks/usePatientTriage.ts	Triage und Atmung warten auf die API. Fehler werden mit console.error protokolliert, aber nicht an den aufrufenden Ablauf weitergegeben. Keine dauerhafte lokale Speicherung der Änderungen.
client-app/src/pages/TriagePage1.tsx	GRÜN/SCHWARZ werden zuerst mit useState angezeigt. Der API-Aufruf folgt ohne eine abgewartete Bestätigung. Patient und Einsatz stammen aus location.state.
client-app/src/pages/TriagePage2.tsx	Bei „Atmung suffizient“ wird der Hook abgewartet; dieser schluckt Fehler. Deshalb folgt trotzdem die Navigation. ROT wird ebenfalls lokal vor Bestätigung gesetzt.
client-app/src/pages/TriagePage3.tsx	ROT/GELB werden lokal gesetzt. GELB sendet blutung=true, ROT blutung=false. Der Text „Atmung suffizient“ ist Bestandteil der Ansicht und kein erneuter Servernachweis.
client-app/src/pages/ShowBodyFront.tsx	Markierungen werden zuerst in useState geändert, dann per PUT gesendet. Fehler nur in der Konsole. Beim Öffnen werden Körperteile per GET geladen; ohne Einträge bleibt die Grafik leer.
client-app/src/pages/ScanPatient.tsx	Erst eine erfolgreiche Serververifikation führt zur Triage mit Patient-ID. QR-Fehler werden hier auch sichtbar gemeldet. Die Entwicklungs-Liste unbenutzter QR-Codes wird vom Server geladen.
client-app/src/stores/authStore.ts	Zustand persistiert Token und Admin-Token unter auth-storage. validateToken ruft /api/validate-token auf; bei Verbindungsfehler false. Gespeicherte Tokens sind keine Offline-Anmeldung.
client-app/src/stores/patientLocationStore.ts	Persistiert jobs ausdrücklich in localStorage. GPS-Aufnahme → Schreiben → geschrieben oder Fehler. Nur aktive Aufträge gelten nach 30 Sekunden als veraltet; ein error-Auftrag wird durch ensurePatientLocation nicht erneut gestartet.
client-app/src/features/ambulanzprotokoll-page1/page1Storage.ts	Liest und schreibt Formularentwürfe als JSON in localStorage: ambdoku:page1:patient:<ID>. Enthält auch Zeitstempelvergleich; beschädigtes JSON liefert beim Lesen null.
client-app/src/features/ambulanzprotokoll-page1/page1Api.ts	GET/PUT /v2/persons/<ID>/ambulanzprotokoll-page1. Die lokale Speicherung und die Serveraufrufe sind getrennte Mechanismen.
Formular-Workflow im Verzeichnis client-app/src/features/ambulanzprotokoll-page1/	Lokales Speichern und Server-Speicherversuch; Ereignisbehandlung für online/offline. Ausstehender Synchronisationszustand ist im Workflow gehalten. Keine allgemeine dauerhafte Warteschlange für Triageänderungen daraus ableitbar.

Codepfad	Relevanter Befund
client-app/vite.config.ts	React, Entwicklungsproxy /api nach localhost:5000, Build nach dist und jsdom-Testumgebung. Kein Service-Worker-/PWA-Plugin in der gezeigten Konfiguration.
client-app/package.json	Scripts für dev, build, lint und test. React, Axios, Zustand und React Query vorhanden; keine explizite IndexedDB-/Workbox-Bibliothek in der gezeigten Paketliste.

Systematische Prüfung der Offline-Mechanismen

Prüfpunkt	Befund / Grenze
indexedDB / serviceWorker	Bei den projektweiten Suchen keine relevanten Treffer; zusätzlich am getesteten Ursprung keine Datenbanken bzw. Registrierungen. Ausgangsannahme in diesem Umfang bestätigt.
localStorage	Nachgewiesen für Auth, GPS-Aufträge und Formularentwürfe. „Kein IndexedDB“ bedeutet nicht „keine lokale Speicherung“.
sessionStorage	Zum Prüfzeitpunkt leer; keine relevante Nutzung in den betrachteten Produktionsdateien nachgewiesen.
navigator.onLine	Bei Serverausfall true. Nicht als zuverlässiger PLS-Status verwendbar.
offline / online	Ereignisbehandlung im Formular-Workflow. Künstlicher online-Test erfolgreich; echte Netzunterbrechung separat ungeprüft.
queue / retry / sync	Keine allgemeine dauerhafte Triage-Warteschlange in den geprüften Dateien. Formular-Synchronisation ist begrenzt vorhanden. Backend-/MQTT-Treffer sind kein Nachweis für Paket C.
API-Fehler	Triage und Körperteile protokollieren überwiegend in der Konsole. QR-Verifikation zeigt zusätzlich eine Fehlermeldung. Fehlende Serverbestätigung ist nicht überall sichtbar.
Stores / Persistenz	Persistenz einzelner Stores vorhanden; Triageentscheidungen liegen in den betrachteten Seiten nur im flüchtigen Ansichtsstatus.

6. Automatisierte Tests

Aus dem Verzeichnis client-app wurden folgende vorhandene Tests ausgeführt:

npm test -- src/features/ambulanzprotokoll-page1/page1Workflow.test.tsx src/pages/AmbulanzprotokollPage1.route.test.tsx src/stores/patientLocationStore.test.ts

Testdatei	Ergebnis	Aussage
page1Workflow.test.tsx	7 Tests bestanden	Vorhandene automatisierte Prüfungen des Formular-Workflows liefen erfolgreich.
AmbulanzprotokollPage1.route.test.tsx	2 Tests bestanden	Patient-/Einsatzkontext beim Route-Einstieg und Wiederherstellung einer lokal gespeicherten Signatur mit Testumgebung geprüft.
patientLocationStore.test.ts	1 Test bestanden	Ein einzelner GPS-Auftrag und dessen erfolgreicher API-Schreibablauf mit Mocks geprüft.
Gesamt	3 Dateien / 10 Tests bestanden	Vitest 4.1.1; Gesamtdauer laut Ausgabe 1,01 Sekunden.

Die ausgewählten Tests bestanden vollständig. Sie verwenden eine Testumgebung mit simulierten Abhängigkeiten und ersetzen keine realen Ausfall- oder Browser-Neustarttests. Ein vollständiger Testlauf des gesamten Projekts, Lint, ein erneuter Gesamtbuild und Backend-Integrationstests sind durch diesen Nachweis nicht abgedeckt.

7. Bewertung und offene Punkte

Das spätere Ziel T2.7 — Offline-Änderung, Browser-Neustart, Erhalt und bestätigte Übertragung nach Wiederverbindung — ist für die geprüfte Triage nicht erfüllt. Bei Formularentwürfen ist dauerhafte lokale Speicherung vorhanden; die Anwendung kann nach einem Neustart ohne erreichbaren Server jedoch nicht erneut geladen werden. Die erfolgreiche Formularübertragung nach Reload darf deshalb nicht als vollständiger Nachweis einer Offline-Triage gewertet werden.

Grenzen der Untersuchung

Nicht vollständig praktisch geprüft wurden Offline-Anmeldung, Kamerascan und neue Patientenerfassung, GPS-Koordinatenaufnahme, Formular-Finalisierung, Körperrückseite, Kartenkacheln, sämtliche Triagevarianten sowie weitere Admin- und Produktionsabläufe. Der GPS-Zugriff war im verwendeten Browser verweigert. Eine vollständige Offline-Abnahme dieser Funktionen ist aus den vorliegenden Ergebnissen nicht ableitbar.

Nach einem Formular-Reload wurde der Einsatzkontext teilweise als n/a angezeigt; nach einem Triage-Reload fehlte die Patient-ID. Diese Beobachtungen betreffen die Wiederherstellung des Anwendungskontexts. Ein Verlust der gespeicherten Patientendaten ist dadurch nicht nachgewiesen.

Fachlich und technisch zu klären

Thema	Offener Punkt
Umfang der nächsten Prüfung	Festlegung der zusätzlich erforderlichen Funktionen, insbesondere weiterer Formular- und Patientenerfassungsabläufe.
Speicherstatus in der Oberfläche	Umgang mit Triagebestätigungen und Körpermarkierungen, deren API-Speicherung fehlschlägt.
Dauerhafte Offline-Speicherung	Berücksichtigung vorhandener localStorage-Entwürfe und GPS-Aufträge bei der späteren IndexedDB-Warteschlange.
Erreichbarkeit und Nachsendung	Prüfung der tatsächlichen PLS-Erreichbarkeit und Auslösung der Übertragung ohne Browser-online-Ereignis.
Patienten- und Einsatzkontext	Wiederherstellung nach Reload und Browser-Neustart; Behandlung eines fehlenden Einsatzkontexts.
GPS-Fehler	Erneuter Versuch nach Fehler oder verweigerter Berechtigung sowie zulässige Speicherung der Positionsdaten.
Bedeutung des Blutungsfelds	Abgleich von PLS „stillbar“ mit „Blutung vorhanden“ im Ambulanzsystem.
Konfliktbehandlung	Verbindliche Regeln für Zeitstempel und unterschiedliche lokale bzw. serverseitige Formularstände.
Abnahmestand und Testdaten	Festlegung des verbindlichen Builds und der zulässigen synthetischen Testdaten für spätere Implementierungs- und Abnahmetests.

8. Grundlagen

Projektauftrag zur PLS–TRIAS Integration: Paket C, Stufe 1, sowie Abgrenzung zur serverseitigen Übertragung in Paket D.

TRIAS: docs/Erlaeuterungen_PLS_Ambulanzsystem.md — Aufgabenbeschreibung, Offline-Ziele und bekannte Stolpersteine.

PLS: README.md, docker-compose.yml und docker-compose.dev.yml — lokaler Betriebsaufbau.

Die in Abschnitt 5 aufgeführten Quellcodedateien und die in Abschnitt 6 genannten Testdateien.

Praktische Browser- und Terminalprüfungen: API-Fehler, Health-Status, Browser-Speicher sowie Kontrolle der gespeicherten Serverwerte.